

# Spring Framework

## Généralités

### I-Introduction

#### Qu'est-ce que le framework Spring ?

Dans cette introduction, nous allons comprendre ce qu'est le framework Spring, quel problème il résout, pourquoi il est devenu si important dans le développement Java et quels avantages il apporte aux applications du monde réel.

Avant l'existence du framework Spring, les développeurs Java construisaient leurs applications de manière très manuelle.

Chaque classe était responsable de la création des objets dont elle avait besoin. Les développeurs devaient gérer eux-mêmes la création des objets, leur connexion, leur configuration et leur cycle de vie.

Cette approche manuelle a créé plusieurs problèmes majeurs.

Premièrement, le code devenait fortement couplé.

Un couplage fort signifie qu'une classe dépend fortement d'une autre classe. Ainsi, si vous modifiez une partie du système, vous devez souvent modifier plusieurs autres parties également. Cela rend les applications difficiles à modifier et encore plus difficiles à faire évoluer.

Deuxièmement, les tests devenaient compliqués, car les objets étaient créés directement dans le code. Il n'était pas facile de remplacer certaines parties du système par des objets simulés (mock objects) pour les tests.

Troisièmement, les projets n'étaient pas bien organisés. Les développeurs finissaient par écrire beaucoup de code répétitif, notamment pour la configuration, l'initialisation et le câblage (liaison) des objets.

Cela ralentissait le développement, augmentait le nombre de bugs et rendait la maintenance des applications coûteuse.

Pour résoudre ces problèmes, le framework Spring a été introduit.

#### Qu'est-ce que le framework Spring ?

Spring est un framework Java qui fournit une manière structurée et efficace de construire des applications.

Il se concentre sur l'aide aux développeurs pour écrire un code propre, organisé et maintenable. Au lieu que les développeurs gèrent tout manuellement, Spring fournit un système bien conçu pour gérer les composants de l'application.

Il standardise la manière dont les objets sont créés, comment ils interagissent et comment ils sont configurés.

En termes simples, Spring s'occupe de la configuration complexe et vous vous concentrez sur l'écriture de la logique métier.

### **Quel problème Spring résout-il ?**

Spring résout le problème de la complexité dans le développement des applications Java.

Dans le Java traditionnel, les développeurs devaient gérer manuellement la création des objets, leur cycle de vie, la configuration, la gestion des dépendances et la structure de l'application.

Spring automatise toutes ces responsabilités. Cela signifie moins d'erreurs, moins de code et davantage de concentration sur la logique réelle de l'application.

Spring résout également le problème du couplage fort. Il encourage une conception où les classes dépendent d'abstractions et non d'implémentations concrètes. Cela rend les applications flexibles et faciles à modifier.

Et en fournissant une structure standard, Spring garantit que même les grandes applications restent organisées, prévisibles et évolutives.

### **Pourquoi Spring est-il important ?**

Spring est devenu important parce qu'il a apporté de la simplicité au développement Java d'entreprise.

Avant Spring, les applications d'entreprise étaient construites avec des technologies lourdes, complexes et difficiles à gérer.

Spring a introduit une conception d'application légère, en couches et facile à gérer.

Il a permis aux développeurs de créer des applications plus rapidement, de les tester facilement et de les maintenir pendant des années.

Pour cette raison, Spring est rapidement devenu un standard dans le monde Java.

Aujourd'hui, si vous travaillez dans le développement back-end en Java, il y a de fortes chances que le projet utilise Spring.

### **Avantages de Spring**

Spring aide les développeurs à écrire un code propre et bien organisé.

Il réduit la complexité en prenant en charge le câblage des objets et la configuration.

Les applications Spring sont plus faciles à tester, plus faciles à étendre et plus faciles à maintenir.

Spring fournit également une structure cohérente entre les projets, ce qui signifie que les membres de l'équipe peuvent comprendre et travailler sur l'application sans confusion.

Spring est aussi très flexible. Vous pouvez créer de petites applications, de grands systèmes d'entreprise ou des microservices distribués. Spring s'adapte à tous ces cas d'utilisation.

En résumé, Spring est un framework Java puissant et largement utilisé qui simplifie le développement d'applications. Il résout les problèmes de couplage fort, de complexité et de conception non structurée. Il est important parce qu'il apporte stabilité, flexibilité et efficacité au développement logiciel. Et il est bénéfique parce qu'il rend votre application propre, testable, évolutive et facile à maintenir.

## II-Fonctionnalités importantes du framework Spring

Dans cette section, nous allons comprendre les fonctionnalités importantes du framework Spring. Ce sont ces fonctionnalités qui font de Spring l'un des frameworks les plus utilisés dans le monde Java. Nous irons étape par étape et nous expliquerons tout avec des exemples simples de la vie réelle.

**La première et la plus importante fonctionnalité de Spring est l'injection de dépendances, également appelée DI.**

L'injection de dépendances signifie simplement que vos classes ne créent pas les objets dont elles dépendent. C'est Spring qui les crée et vous les fournit. Prenons un exemple simple de la vie réelle.

En d'autres termes, Spring fournit ce dont une classe a besoin au lieu que la classe le crée elle-même.

L'injection de dépendances signifie donc que vos classes ne créent pas les objets dont elles ont besoin. À la place, Spring crée ces objets pour vous et les injecte là où c'est nécessaire.

Cela supprime la nécessité d'utiliser le mot-clé **new** partout. Cela rend votre code plus propre, mieux organisé et plus facile à tester.

**La deuxième fonctionnalité est l'Inversion de Contrôle ou IoC.**

Normalement, en Java, votre code est responsable de la création des objets et de l'appel de leurs méthodes. Mais avec Spring, ce contrôle est inversé. C'est le framework Spring qui contrôle la création et la gestion des objets.

Considérez Spring comme un chef en cuisine. Vous passez simplement une commande : « J'ai besoin de cet objet », et Spring le prépare, le gère et vous le fournit.

Votre code ne contrôle plus la création des objets. C'est Spring qui s'en charge. C'est pourquoi on appelle cela l'Inversion de Contrôle.

### Gestion des beans

Les objets que Spring crée sont appelés des **beans**. Spring stocke et gère ces beans dans ce que l'on appelle le **conteneur Spring (Spring Container)**.

Vous pouvez imaginer le conteneur Spring comme un entrepôt. Tous vos objets importants sont stockés dans cet entrepôt, et chaque fois que votre application en a besoin, Spring les fournit.

Cela signifie que vous ne vous préoccupez pas de la manière dont les objets sont créés ni de leur durée de vie. Spring gère tout.

### Configuration basée sur les annotations

Auparavant, Spring nécessitait de grands fichiers de configuration XML, mais aujourd'hui nous utilisons des annotations simples.

Des annotations comme **@Component**, **@Service**, **@Repository** et **@Autowired** indiquent à Spring quelles classes gérer et où injecter les dépendances.

Cela rend le développement avec Spring beaucoup plus rapide, plus propre et plus simple.

### Un autre grand avantage de Spring est le couplage faible.

Le couplage faible signifie que les objets dépendent d'interfaces et non de classes concrètes.

Par exemple, si votre application doit envoyer des notifications, elle peut dépendre d'une interface **NotificationService**, et non d'une implémentation spécifique comme **Email** ou **SMS**.

Plus tard, si l'entreprise décide de ne plus utiliser les SMS et de passer aux notifications **WhatsApp**, vous changez simplement l'implémentation, sans modifier toute l'application.

Cette flexibilité est la raison pour laquelle Spring est utilisé dans les applications de niveau entreprise.

### Tests unitaires faciles

puisque Spring fournit automatiquement les dépendances, nous pouvons facilement remplacer les objets réels par des objets simulés (mock objects) pendant les tests.

Cela signifie que les tests unitaires deviennent plus rapides et plus propres.

Vous n'avez pas à modifier la logique de votre application uniquement pour la tester. Spring prend naturellement en charge les workflows de test.

### Configuration centralisée et cohérente.

Spring vous permet de regrouper toute votre configuration en un seul endroit, qu'il s'agisse des paramètres de base de données, du câblage des services ou des valeurs spécifiques à l'environnement.

Spring fournit une approche de configuration centralisée afin que votre projet reste organisé et facile à gérer.

### Léger et modulaire.

Même si Spring peut faire beaucoup de choses, il reste léger et modulaire.

Cela signifie que vous n'utilisez que les parties dont votre application a réellement besoin. Vous n'êtes pas obligé de charger tout le framework.

Par exemple, si vous avez uniquement besoin de l'injection de dépendances, vous pouvez utiliser **Spring Core** et ignorer le reste.

Cela rend votre application efficace et optimisée.

## Support déclaratif.

Spring vous permet d'activer des fonctionnalités comme les transactions, le cache ou la sécurité simplement en utilisant des annotations simples.

Par exemple, au lieu d'écrire un code complexe pour gérer les transactions, vous pouvez simplement ajouter **@Transactional** et Spring se chargera automatiquement de la logique de commit et de rollback.

Cela réduit le code répétitif et évite les bugs.

Résumons rapidement les fonctionnalités importantes du framework Spring.

**Injection de dépendances** : Spring fournit automatiquement les objets nécessaires.

**Inversion de contrôle** : Spring contrôle la création des objets au lieu de votre code.

**Gestion des beans** : Spring stocke et gère les objets dans le conteneur Spring.

**Configuration basée sur les annotations** : Pas de XML. Des annotations simples et propres.

**Couplage faible** : Facile de changer les implémentations sans casser le code.

**Tests faciles** : Les mocks peuvent être injectés facilement.

**Configuration centralisée** : Garde votre application organisée, légère et modulaire.

**Utilisez uniquement ce dont vous avez besoin** : Rien de superflu.

**Support déclaratif supplémentaire** : Utilisez des annotations pour activer des fonctionnalités clés comme les transactions, le cache et la sécurité.

Spring se concentre sur l'aide aux développeurs pour écrire la logique métier, tandis que Spring gère la configuration complexe, le câblage et le cycle de vie en arrière-plan. C'est ce qui rend Spring si puissant, flexible et largement utilisé.

### III- Modules importants du framework Spring

Dans cette section, nous allons explorer les principaux modules du framework Spring et comprendre le rôle de chacun ainsi que les problèmes qu'ils résolvent dans les applications réelles.

Spring n'est pas une seule entité ; c'est un ensemble de différents modules, et chaque module se concentre sur la résolution d'un ensemble spécifique de problèmes liés au développement d'applications.

La beauté de Spring réside dans sa modularité. Cela signifie que vous n'êtes pas obligé d'utiliser tout le framework. Vous choisissez simplement les modules dont votre projet a besoin.

Passons en revue les modules importants un par un.

#### Spring Core.

C'est la fondation de tout le framework Spring. Le module Spring Core fournit la fonctionnalité d'injection de dépendances. Cela signifie que Spring prend la responsabilité de créer les objets, de les fournir à d'autres objets et de gérer leur cycle de vie.

Avant Spring, les développeurs devaient créer et gérer manuellement les objets en utilisant le mot-clé **new**, ce qui entraînait un couplage fort et rendait les applications difficiles à maintenir et à tester.

Spring Core résout ce problème en introduisant l'inversion de contrôle. C'est le framework qui prend le contrôle de la création des objets, et non le développeur.

Ainsi, lorsque quelqu'un mentionne Spring, il fait généralement référence au module **Spring Core** et au système d'injection de dépendances.

#### Spring Context.

Vous pouvez considérer ce module comme une extension du module **Spring Core**. Le module Spring Context gère et organise tous les beans Spring. Il fournit ce que l'on appelle le **contexte d'application (ApplicationContext)**, qui est le cœur de votre application Spring.

Le problème qu'il résout est celui de la visibilité et de l'accessibilité.

Votre application peut contenir des centaines de beans.

Spring Context les répertorie et fournit le bon bean chaque fois que votre code en a besoin. C'est comme une salle de stockage de bureau bien organisée, où tout est étiqueté et facile à trouver.

**Spring AOP**, qui signifie *Aspect Oriented Programming* (programmation orientée aspects).

AOP résout un problème très courant dans presque tous les projets : la répétition de la même logique à différents endroits.

Par exemple : enregistrer chaque requête (logging), valider l'authentification, mesurer les performances ou appliquer des contrôles de sécurité.

Sans AOP, vous devriez écrire le même code encore et encore dans plusieurs classes.

Cela entraîne du code dupliqué et augmente les risques d'erreurs.

Spring AOP vous permet d'écrire cette logique commune une seule fois, et Spring l'appliquera automatiquement partout où c'est nécessaire.

C'est comme installer des caméras de surveillance dans un bâtiment. Vous n'avez pas besoin d'engager un agent de sécurité pour chaque pièce. Un seul système surveille tout.

### Spring JDBC.

JDBC signifie *Java Database Connectivity*. Avant Spring, travailler avec des bases de données en Java nécessitait beaucoup de code répétitif.

Vous deviez ouvrir manuellement les connexions, préparer les requêtes, exécuter les instructions SQL, gérer les exceptions, puis tout fermer soigneusement.

Même une petite requête de base de données pouvait nécessiter 30 à 40 lignes de code.

Spring JDBC résout ce problème en fournissant des classes **JdbcTemplate**, où la plupart du code répétitif est déjà géré pour vous.

Vous vous concentrez simplement sur l'écriture de la logique SQL réelle. Tout le reste est pris en charge par Spring.

**Spring ORM.** ORM signifie *Object Relational Mapping* (mappage objet-relationnel).

Spring ORM intègre Spring avec des frameworks ORM comme **Hibernate** et **JPA**.

Le problème qu'il résout est le décalage entre les tables de base de données et les objets Java.

Sans ORM, les développeurs devaient convertir manuellement les données des tables SQL en objets Java, puis reconvertir les objets Java en données SQL. Cela était source d'erreurs et prenait beaucoup de temps.

Spring ORM permet à votre application d'interagir avec la base de données en termes d'objets, et non directement en SQL.

Cela rend l'accès aux données beaucoup plus simple et plus propre.

### Spring Transaction Management.



Une transaction signifie exécuter plusieurs étapes où soit toutes les étapes réussissent, soit aucune ne réussit.

Par exemple, le transfert d'argent entre deux comptes bancaires. L'argent doit être débité d'un compte et crédité sur un autre. Si une étape échoue, les deux opérations doivent être annulées (rollback).

Avant Spring, les développeurs devaient écrire manuellement les instructions de **commit** et de **rollback**.

Spring Transaction Management vous permet simplement d'annoter une méthode avec **@Transactional**, et Spring gèrera automatiquement le commit et le rollback.

Cela réduit les bugs et garantit la cohérence des données.

### Spring MVC.

Ce module est utilisé pour construire des applications web et des services web RESTful.

Avant Spring MVC, la gestion des requêtes web en Java nécessitait l'utilisation de servlets et beaucoup de code boilerplate complexe.

Spring MVC simplifie cela en utilisant des classes contrôleurs, des méthodes pour gérer les requêtes HTTP et une conversion automatique en JSON pour les services web RESTful.

C'est aujourd'hui le module le plus couramment utilisé pour développer des services back-end.

### Spring Security

La sécurité est une exigence essentielle dans presque toutes les applications modernes.

Nous avons besoin de connexions (login), de rôles utilisateurs, de permissions, de gestion des sessions et de protection contre les attaques.

Mettre tout cela en œuvre manuellement est extrêmement difficile et risqué.

Spring Security fournit l'authentification et l'autorisation prêtes à l'emploi.

Il protège également les applications contre les menaces de sécurité courantes.

Ce module garantit que votre application est sécurisée et bien contrôlée.

### Spring Cloud

Spring Cloud fournit des outils pour développer et gérer des applications distribuées et des architectures microservices.

Il offre des fonctionnalités comme la découverte de services (Service Discovery), la configuration centralisée, l'équilibrage de charge, la gestion des API Gateway et la tolérance

aux pannes, afin de simplifier le déploiement et la communication entre services dans des environnements cloud.

## Spring Boot

Spring Boot n'est pas un remplacement de Spring. C'est une couche pratique au-dessus de Spring.

Il supprime les configurations complexes et nous aide à construire rapidement des applications Spring.

Considérez Spring Boot comme la version automatique de Spring : moins de configuration, moins de fichiers de paramétrage et un développement beaucoup plus rapide.

### Résumé

Pour résumer, Spring est composé de différents modules, et chaque module résout un problème réel de développement.

**Spring Core** et Spring Context gèrent la création des objets.

**Spring AOP** gère la logique transversale (cross-cutting concerns).

**Spring JDBC et ORM** simplifient l'accès à la base de données.

**Spring Transaction** garantit des opérations de données sécurisées avec **@Transactional**.

**Spring MVC** permet de construire des API et des applications web.

**Spring Security** protège l'application.

**Spring Cloud** permet de développer des applications distribuées et des architectures microservices.

**Spring Boot** rend tout plus simple et plus rapide.

Cette architecture modulaire est ce qui rend Spring puissant, flexible et largement utilisé dans les projets réels.

